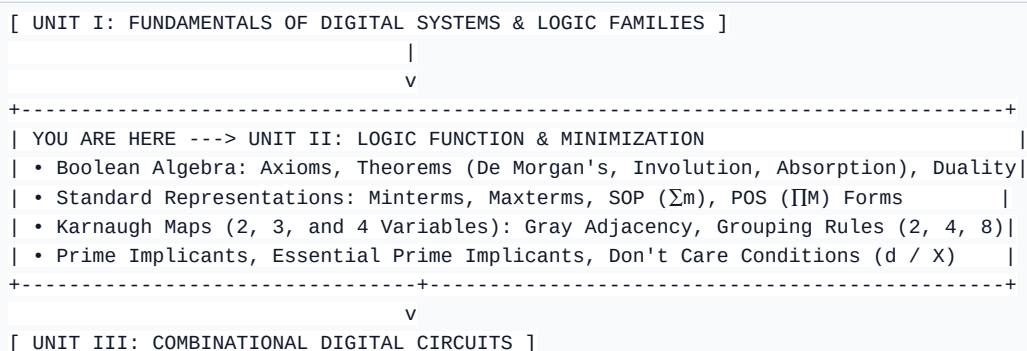


DEC Unit II: Logic Function & Minimization

Comprehensive First-Principles Lecture Notes

Unit II: Logic Function and Minimization

Master Subject Map: Unit II Position



In 3 lines: In Unit I, we learned how logic gates operate physically. But when designing a real digital system, naive designs require dozens of redundant gates. Unit II provides the mathematical machinery (Boolean Algebra) and graphical spatial tools (Karnaugh Maps) to strip away every redundant transistor, yielding the cheapest, fastest possible circuit.

Prerequisites for this Unit

Before studying Unit II, ensure you recall these concepts from earlier sections: 1. **Basic Gates (Unit I, Section 1.5):** AND ($X = AB$), OR ($X = A + B$), and NOT ($X = \overline{A}$). 2. **Truth Tables (Prereq 6):** An exhaustive list of outputs for all 2^n combinations of n binary inputs. 3. **Powers of Two (Prereq 6):** **1, 2, 4, 8, 16** (governs the size of allowed K-map grouping loops). 4. **Gray Code Adjacency (Unit I, Section 1.4):** Consecutive codes differ by only a single bit (**00, 01, 11, 10**).

PART 1: Boolean Algebra Axioms, Theorems, and Standard Forms

2.1 The Postulates and Axioms of Boolean Algebra

Source: Kumar (Ch. 5, pp. 207–211); Mano (Ch. 2, pp. 58–64)

[Pacing: Pure Foundations — Read Deliberately to Internalize Rules]

Need (Problem-First)

In high-school ordinary algebra, variables represent real numbers and arithmetic follows standard operations ($1 + 1 = 2$). But electronic switches only have two states: **0** and **1**. Applying ordinary algebra to switches produces nonsense (for example, in ordinary math, $1 + 1 = 2$, but in an OR gate, turning on switch A and switch B simply keeps the bulb ON: $1 + 1 = 1$). We need a rigorous mathematical system specifically tailored for two-valued logic. This is **Boolean Algebra**, formulated by George Boole in 1854 and adapted to switching circuits by Claude Shannon in 1938.

Chain of Cause and Effect

Switches are two-valued $\rightarrow$ mathematicians define formal closure under OR (+) and AND ($\cdot$) operations $\rightarrow$ axioms establish identity, commutativity, distributivity, and complements $\rightarrow$ these axioms permit rigorous mathematical simplification of electronic

hardware.

The Huntington Postulates (Axioms) Defined

A set of elements $B = \{0, 1\}$ together with two binary operators $+$ (logical OR) and $\cdot$ (logical AND) satisfies the following axioms (Kumar, p. 208; Mano, p. 60):

1. **Closure:** For every A, B in B : $A + B$ in B (the result of ORing two bits is always a bit). $A \cdot B$ in B (the result of ANDing two bits is always a bit).
2. **Identity Elements:** - Identity for OR is 0 : $A + 0 = A$. (ORing with 0 does not change the signal). - Identity for AND is 1 : $A \cdot 1 = A$. (ANDing with 1 preserves the signal).
3. **Commutativity:** $A + B = B + A$ $A \cdot B = B \cdot A$ (The order of physical input wires to a gate does not alter the output).
4. **Distributivity:** - AND distributes over OR: $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$. - OR distributes over AND (Crucial Boolean Special Rule!):

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

Note for beginners: This second distributive law is **false in ordinary high-school algebra**, but **100% TRUE in Boolean algebra!**

5. **Complement (Inverse):** For every element A in B , there exists a unique element A in B such that: $A + A' = 1$ (A wire ORed with its inverse is always connected to $+5V$). $A \cdot A' = 0$ (A wire ANDed with its inverse can never be closed; it is always $0V$).

2.2 Core Theorems of Boolean Algebra and De Morgan's Laws

Source: Kumar (Ch. 5, pp. 211–219); Mano (Ch. 2, pp. 64–67)

[Pacing: Conceptually Critical — Memorize and Understand These Transformation Tools]

Need (Problem-First)

When an engineer writes down the raw logic equation for a real-world controller, the equation often contains ten or twenty terms. If built directly, the circuit would require dozens of integrated circuit packages. We need rigorous mathematical theorems to cancel out redundant terms on paper before soldering chips.

The Core Theorems

1. **Idempotent Laws (Kumar, p. 211):**

$$A + A = A \quad A \cdot A = A$$

Physical Meaning: Tying two inputs of an OR or AND gate to the exact same wire produces the input itself.

2. **Boundedness (Null / Annihilation) Laws (Kumar, p. 212):**

$$A + 1 = 1 \quad A \cdot 0 = 0$$

Physical Meaning: An OR gate with one input tied permanently to $+5V$ is permanently locked HIGH (1). An AND gate with one input tied to Ground ($0V$) is permanently locked LOW (0).

3. **Involution (Double Complement) (Kumar, p. 212):**

$$A = A''$$

Physical Meaning: Two inverters in series cancel each other out.

4. **Absorption Laws (Kumar, p. 213):**

$$A + (A \cdot B) = A \quad A \cdot (A + B) = A$$

Conceptual Proof of $A + AB = A$:

$$\text{Step 1: Factor using identity axiom } (A = A \cdot 1): A + AB = A \cdot 1 + A \cdot B$$

$$\text{Step 2: Apply distributive law: } = A \cdot (1 + B)$$

$$\text{Step 3: Apply boundedness law } (1 + B = 1): = A \cdot (1)$$

$$\text{Step 4: Apply identity law: } = A$$

Physical Meaning: If the condition A is already asserted, the term AB is completely redundant hardware that can be cut from the circuit board!

5. **Redundant Literal Rule (Consensus Variant) (Kumar, p. 214):**

$$A + AB = A + B$$

Conceptual Proof:

Step 1: Apply Boolean distributive law ($A + BC = (A+B)(A+C)$): $A + AB = (A + A) \cdot (A + B)$

Step 2: Apply complement law ($A + A = 1$): $= 1 \cdot (A + B)$

Step 3: Apply identity law: $= A + B$

Physical Hardware Reality: The **A** term was completely useless!

6. De Morgan's Theorems

Source: Kumar (Ch. 5, pp. 215–218); Mano (Ch. 2, pp. 65–67)

[Pacing: Slow & Deliberate — The Most Important Theorem in Digital Electronics]

Theorem 1: Complementation of a Product (The NAND Theorem)

"The complement of a product of variables is equal to the sum of their individual complements" (Kumar, p. 215):

$$A \cdot B = A + B$$

Physical Gate Meaning: A **NAND gate** is logically identical to an **OR gate with inverted inputs** (a Bubbled OR gate).

Theorem 2: Complementation of a Sum (The NOR Theorem)

"The complement of a sum of variables is equal to the product of their individual complements" (Kumar, p. 216):

$$A + B = A \cdot B$$

Physical Gate Meaning: A **NOR gate** is logically identical to an **AND gate with inverted inputs** (a Bubbled AND gate).

Mathematical Verification by Truth Table:

Let us prove Theorem 1 ($AB = A + B$) exhaustively across all four possible input combinations:

A	B	AB	AB (LHS)	A	B	A + B (RHS)	Match?
0	0	0	1	1	1	1	Yes
0	1	0	1	1	0	1	Yes
1	0	0	1	0	1	1	Yes
1	1	1	0	0	0	0	Yes

Every single row of Column 4 (**AB**) matches Column 7 (**A + B**) exactly. The theorem is proven beyond doubt.

2.3 The Principle of Duality

Source: Kumar (Ch. 5, p. 210); Mano (Ch. 2, p. 64)

[Pacing: Fast & Methodical — A Powerful Symmetry Tool]

Need & How It Works

The **Duality Principle** states that any true Boolean algebraic identity remains true if you perform three simultaneous swaps (Kumar, p. 210): 1. Change every OR operator (**+**) to an AND operator (**·**). 2. Change every AND operator (**·**) to an OR operator (**+**). 3. Change every identity **0** to **1**, and every **1** to **0**. (Leave variable literals **A**, **B**, **C** uncomplemented!) - Example: The dual of the identity $A + A = 1$ is:

$$A \cdot A = 0$$

This halves the effort required in formal mathematical proofs.

2.4 Standard Representations: Minterms, Maxterms, SOP, and POS

Source: Kumar (Ch. 5, pp. 220–228); Mano (Ch. 2, pp. 73–82)

[Pacing: Systematic Bookkeeping — Foundation for K-Maps]

Need (Problem-First)

A Boolean function can be written in dozens of arbitrary algebraic forms (e.g., $F = AB + C(D + E)$ vs $F = (A + C)(B + C)D + CE$). To communicate unambiguously, build systematic automated tools, and enter logic into Karnaugh Maps, engineers require standardized **canonical forms**.

Definitions: Minterms vs. Maxterms

1. Minterms (Standard Product Terms) (Kumar, p. 220)

A **minterm** is an AND product of all n literals in the function, where each variable appears exactly once (either in unprimed form A or primed form $\bar{A}$). - **The Rule for Minterms:** A minterm is defined such that it equals **1 for exactly one specific combination of input variables**. - If variable $A = 1$, write it uncomplemented: A . - If variable $A = 0$, write it complemented: $\bar{A}$. - *Designation:* Lowercase m_i , where index i is the decimal equivalent of the binary row. - For 3 variables (A, B, C): - Row 0 (000_2): $m_0 = \bar{A} \bar{B} \bar{C}$ - Row 1 (001_2): $m_1 = \bar{A} \bar{B} C$ - Row 5 (101_2): $m_5 = A \bar{B} C$ - Row 7 (111_2): $m_7 = ABC$

2. Maxterms (Standard Sum Terms) (Kumar, p. 222)

A **maxterm** is an OR sum of all n literals, where each variable appears exactly once. - **The Rule for Maxterms:** A maxterm is defined such that it equals **0 for exactly one specific combination of input variables**. - If variable $A = 0$, write it uncomplemented: A . - If variable $A = 1$, write it complemented: $\bar{A}$. (*Notice this is the exact opposite of minterm convention!*) - *Designation:* Uppercase M_i , where index i is the decimal row number. - For 3 variables (A, B, C): - Row 0 (000_2): $M_0 = A + B + C$ - Row 5 (101_2): $M_5 = \bar{A} + \bar{B} + \bar{C}$ - Row 7 (111_2): $M_7 = \bar{A} + \bar{B} + \bar{C}$

3. Canonical Sum-of-Products (SOP) Form ($\sum m$)

The canonical SOP form expresses a logic function as the logical OR (sum) of all the minterms for which the output function equals 1:

$$F(A, B, C) = \sum m(1, 4, 5, 7) = m_1 + m_4 + m_5 + m_7 = \bar{A} \bar{B} C + A \bar{B} \bar{C} + A \bar{B} C + ABC$$

4. Canonical Product-of-Sums (POS) Form (prod M)

The canonical POS form expresses a logic function as the logical AND (product) of all the maxterms for which the output function equals 0:

$$F(A, B, C) = \text{prod } M(0, 2, 3, 6) = M_0 \cdot M_2 \cdot M_3 \cdot M_6 = (A + B + C)(A + \bar{B} + \bar{C})(\bar{A} + B + \bar{C})(\bar{A} + \bar{B} + C)$$

The Fundamental Conversion Rule (Kumar, p. 226):

The minterms where $F = 1$ and the maxterms where $F = 0$ are exact complements of the full universe of states:

$$\sum m(1, 4, 5, 7) \text{ equiv } \text{prod } M(0, 2, 3, 6)$$

Any function given in SOP can be instantly converted to POS by simply listing the **missing index numbers**!

PART 2: Karnaugh Maps (K-Maps) and Logic Simplification

2.5 The Karnaugh Map (K-Map) Philosophy and Gray-Code Adjacency

[Pacing: Conceptually Critical — The Core Tool for Minimization]

Need (Problem-First)

Minimizing logic functions using Boolean algebra theorems by hand is like doing high-wire acrobatics without a safety net: 1. It is **non-systematic**: there is no fixed algorithm telling you which theorem to apply next. 2. It relies heavily on human pattern-recognition and intuition. 3. It is notoriously **error-prone**: missing a single prime or bar destroys the entire circuit.

We need a **foolproof graphical method** where simplification happens automatically through simple visual pattern grouping. That method is the **Karnaugh Map (K-map)**, invented by Maurice Karnaugh in 1953.

The Core Mechanism: Gray Code Spatial Adjacency

A K-map is a diagram made up of squares, where each square represents one specific minterm (Kumar, p. 263).

The profound genius of the K-map lies in its coordinate labeling: **the rows and columns are arranged in Gray code sequence (00, 01, 11, 10)**, NEVER standard binary (00, 01, 10, 11)! - Why? In Gray code, **adjacent squares differ by only ONE bit variable**. - When two adjacent squares both contain a 1, they represent two minterms that are identical except for one variable (e.g., $\overline{A}B$ and AB). - By the Boolean theorem $\overline{A}B + AB = A(B + \overline{B}) = A(1) = A$, combining two adjacent squares **physically eliminates the variable that changes**!

2.6 Two-Variable and Three-Variable K-Maps

1. Two-Variable K-Map (Kumar, pp. 265–267)

A 2-variable function $F(A, B)$ has $2^2 = 4$ minterms, arranged in a 2×2 grid:

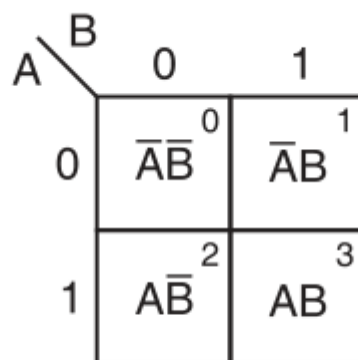


Figure 6.1 The minterms of a two-variable K-map.

Figure

6.1: Format and minterm cell assignments for a two-variable Karnaugh Map. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p.

265 (printed p. 233).

- **Allowed Groupings in 2-Variable K-Map (Kumar, p. 266, Figure 6.4):**
- An isolated single cell ($2^0 = 1$ cell): yields a 2-variable term.
- A pair of adjacent cells ($2^1 = 2$ cells): eliminates 1 variable, yielding a 1-variable term.
- A block of four cells ($2^2 = 4$ cells): eliminates 2 variables, yielding the constant logic 1.

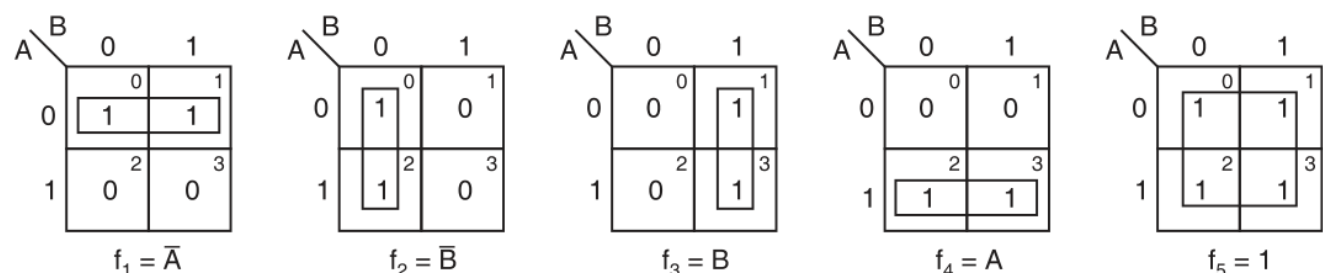


Figure 6.4 The possible minterm groupings in a two-variable K-map.

Figure

6.4: Possible minterm groupings (isolated cell, 2-cell pair, and 4-cell full map) in a two-variable K-map. Sourced from A. Anand Kumar, Fundamentals of

2. Three-Variable K-Map (Kumar, pp. 269–275)

A 3-variable function $F(A, B, C)$ has $2^3 = 8$ minterms, arranged in a 2×4 grid (Kumar, Figure 6.10):

SHOWN IN THE FIGURE.

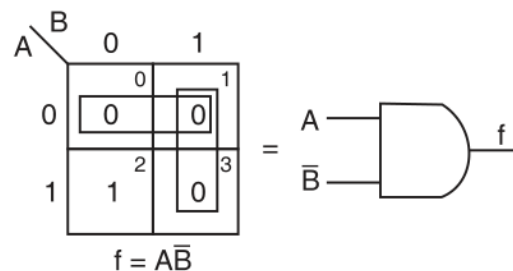


Figure 6.9 Example 6.4: K-map in POS form and logic diagram.

In the given expression, the maxterm M_2 is absent. This is indicated by a 1 on the K-map. The corresponding SOP expression is $\sum m_2$ or $A\bar{B}$. This realization is the same as that for POS form.

6.3 THREE-VARIABLE K-MAP

A function in three variables (A, B, C) expressed in the standard SOP form can have eight possible combinations: $\bar{A}\bar{B}\bar{C}$, $\bar{A}\bar{B}C$, $\bar{A}B\bar{C}$, $\bar{A}BC$, $A\bar{B}\bar{C}$, $A\bar{B}C$, $AB\bar{C}$, and ABC . Each one of these combinations designated by $m_0, m_1, m_2, m_3, m_4, m_5, m_6$, and m_7 , respectively, is called a minterm. A is the MSB of the minterm designator and C is the LSB.

In the standard POS form, the eight possible combinations are: $A + B + C$, $A + B + \bar{C}$, $A + \bar{B} + C$, $A + \bar{B} + \bar{C}$, $\bar{A} + B + C$, $\bar{A} + B + \bar{C}$, $\bar{A} + \bar{B} + C$, and $\bar{A} + \bar{B} + \bar{C}$. Each one of these combinations designated by $M_0, M_1, M_2, M_3, M_4, M_5, M_6$, and M_7 , respectively, is called a maxterm. A is the MSB of the maxterm designator and C is the LSB.

A three-variable K-map has, therefore, $8 (= 2^3)$ squares or cells, and each square on the map represents a minterm or maxterm as shown in Figure 6.10a and Figure 6.10b. The small number in the top right corner of each cell indicates the minterm or maxterm designation.

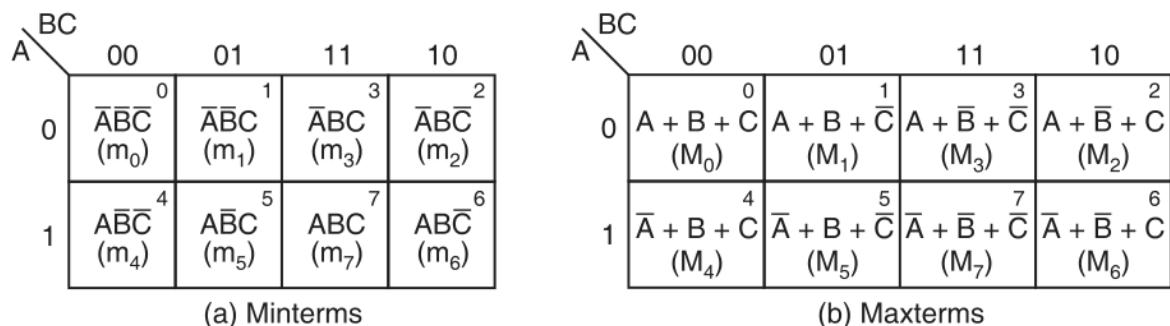


Figure 6.10 The three-variable K-map.

Figure

6.10: Three-variable Karnaugh map structure and minterm cell numbering showing Gray code column sequence (00, 01, 11, 10). Sourced from A. Anand

Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 269 (printed p. 237).

- **Notice the Column Reversal:** The column sequence is **00, 01, 11, 10**. Therefore, the cell numbers in the top row are **0, 1, 3, 2** (cells 3 and 2 are swapped!), and in the bottom row are **4, 5, 7, 6**! This is the most

common place beginners make arithmetic slip-ups.

- **Topological Torus (Wrap-Around Adjacency):** The extreme left column (00) is physically adjacent to the extreme right column (10)! A grouping can roll over the outer edges:

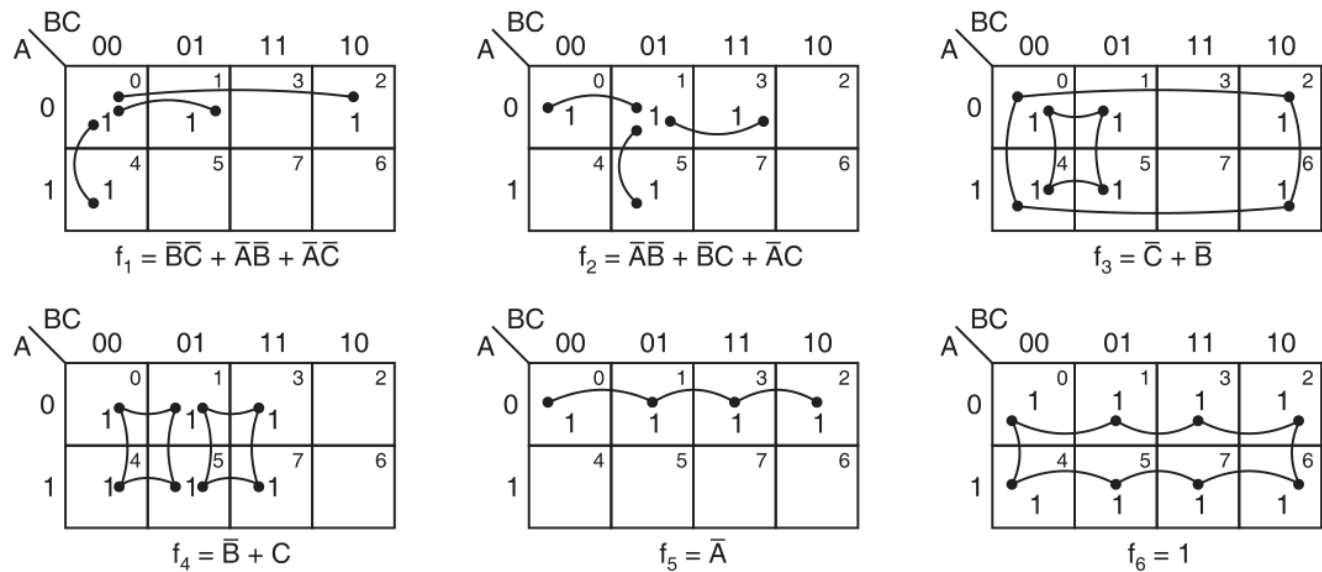


Figure 6.13 Some possible combinations of minterms in a three-variable K-map (SOP form).

Figure

6.13: Valid grouping patterns in a three-variable K-map (singletons, pairs, quads, and end-wrap combinations). Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 272 (printed p. 240).

2.7 The Four-Variable K-Map (Up to 4 Variables)

Source: Kumar (Ch. 6, pp. 276–284); Mano (Ch. 3, pp. 107–112)

[Pacing: The Core Workhorse of Logic Minimization — Study Carefully]

Structure and Geometry

A 4-variable function $F(A, B, C, D)$ has $2^4 = 16$ minterms, arranged in a symmetrical 4×4 grid (Kumar, p. 276, Figure 6.19):

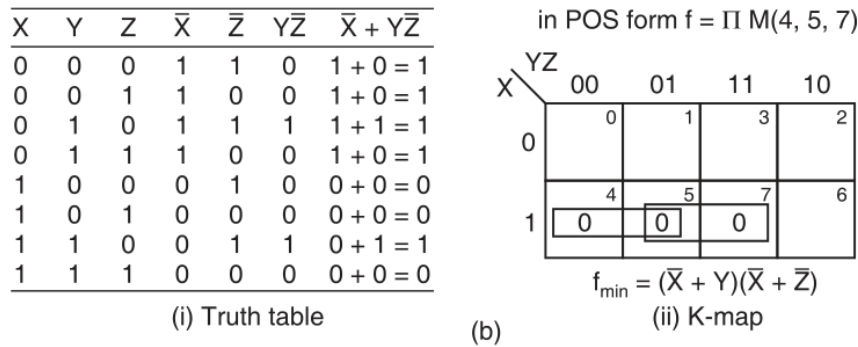


Figure 6.18 Example 6.10.

6.4 FOUR-VARIABLE K-MAP

A four-variable (A, B, C, D) expression can have $2^4 = 16$ possible combinations of input variables such as $\bar{A}\bar{B}\bar{C}\bar{D}$, $\bar{A}\bar{B}\bar{C}D$, ..., $ABCD$ with minterm designations $m_0, m_1, \dots, m_{15}$, respectively, in the SOP form and as $A + B + C + D$, $A + B + C + \bar{D}$, ..., $\bar{A} + \bar{B} + \bar{C} + \bar{D}$ with maxterm designations $M_0, M_1, \dots, M_{15}$, respectively, in the POS form. A four-variable K-map has $2^4 = 16$ squares or cells and each square on the map represents either a minterm or a maxterm as shown in Figure 6.19.

The binary number designations of the rows and columns are in the Gray code. Here 11 follows 01 and 10 follows 11. This is called *adjacency ordering*. The binary numbers along the top of the map indicate the conditions of C and D along any column and binary numbers along the left side indicate the conditions of A and B along any row. The numbers in the top right corners of the squares indicate the minterm or maxterm designations as usual.

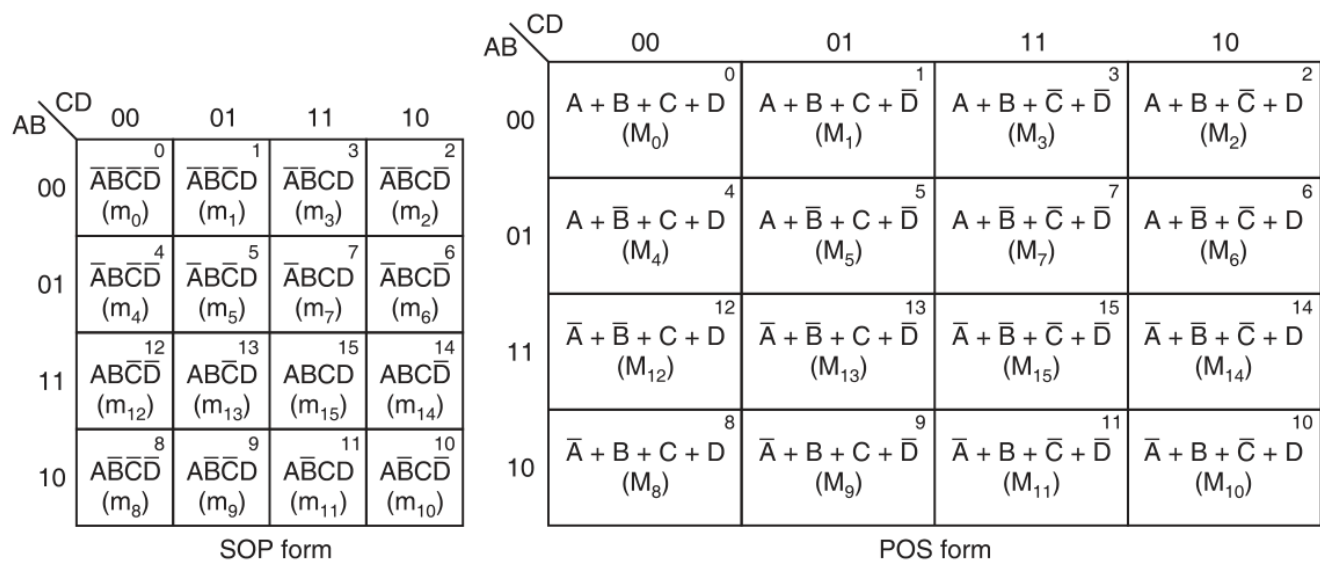


Figure 6.19 The minterms and maxterms of a four-variable K-map.

Figure

6.19: Four-variable Karnaugh Map layout showing row and column Gray code sequences and corresponding minterm / maxterm decimal indices. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 276 (printed p. 244).

The Universal 16-Cell Index Map:

	CD			
AB	00	01	11	10
00	+-----+			
	0	1	3	2
01	+-----+			

		4		5		7		6		
11	+-----+-----+-----+-----+									
		12		13		15		14		<-- Row 11 is row index 12-15!
10	+-----+-----+-----+-----+									
		8		9		11		10		<-- Row 10 is row index 8-11!
	+-----+-----+-----+-----+									

Crucial Double Swap Warning: Notice that both the 3rd column (**CD=11**) and the 3rd row (**AB=11**) are swapped with their 4th counterparts to preserve Gray-code single-bit transitions!

The Rigorous Step-by-Step K-Map Minimization Algorithm:

- Plot the Function:** Place a 1 in every cell corresponding to a minterm of the function. Leave all other cells as 0 (or blank).
- Identify Isolated Cells:** Any 1 that has no adjacent 1s cannot be combined. It must be circled as a 1-cell group (2^0).
- Form the Largest Possible Groups (Powers of Two Only):**
- Group of 16 (2^4):** Covers entire map ■ **F = 1** (eliminates 4 variables).
- Group of 8 (Octet, 2^3):** Eliminates 3 variables, leaving a 1-variable term.
- Group of 4 (Quad, 2^2):** Eliminates 2 variables, leaving a 2-variable term.
- Group of 2 (Pair, 2^1):** Eliminates 1 variable, leaving a 3-variable term.
- Wrap-Around Adjacencies (The 3D Cylindrical Surface):**
- The top row (**AB=00**) is adjacent to the bottom row (**AB=10**).
- The left column (**CD=00**) is adjacent to the right column (**CD=10**).
- The Four Corners:** Cells (**0, 2, 8, 10**) form a single valid **Quad** ($2^2 = 4$ cells)!
- Eliminate Redundant Groups:** Every 1 must be covered at least once. If a group contains *only* 1s that are already covered by other necessary groups, that group is completely redundant and must be erased!

2.8 Prime Implicants and Essential Prime Implicants

Source: Kumar (Ch. 6, pp. 280–283); Mano (Ch. 3, pp. 109–112)

[Pacing: Conceptually Rigorous — Key for Formal Engineering Exams]

Definitions Defined Inline:

- Implicant:** Any single minterm or valid grouping of minterms (of size **1, 2, 4, 8 dots**) for which the output is **1**.
- Prime Implicant (PI):** A rectangle of **1s** (size 2^k) that **cannot be merged into any larger rectangle** (Kumar, p. 281). It is maximal.
- Essential Prime Implicant (EPI):** A Prime Implicant that contains **at least one 1 that is not covered by any other prime implicant** (Kumar, p. 281). **Every EPI must appear in the final minimal expression!**
- Redundant Prime Implicant:** A prime implicant whose **1s** are all completely covered by essential prime implicants. It must be discarded.

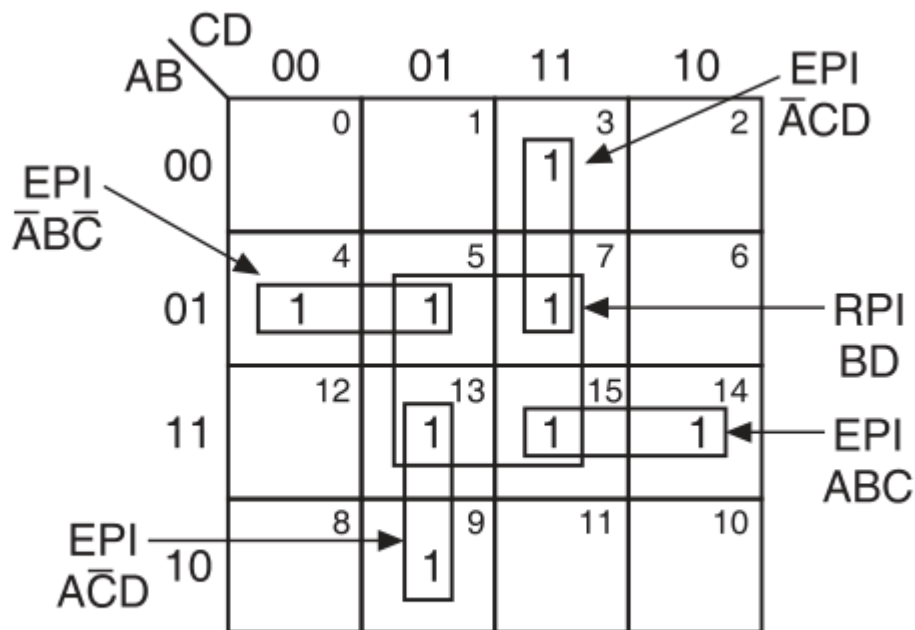


Figure 6.24 Essential and redundant prime implicants.

Figure

6.24: Distinguishing Essential Prime Implicants from Redundant Prime Implicants on a Karnaugh map. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 281 (printed p. 249).

Concrete Fully-Worked 4-Variable Example

Minimize: $F(A, B, C, D) = \sum m(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$ - **Step 1 (Plotting)**: Mark cells 0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14 with 1. - **Step 2 (Identify Groups)**: - Look at columns 1 & 2 (where $CD = 00$ and 01): cells (0, 1, 4, 5, 12, 13, 8, 9) form a massive **Octet (8 cells)** spanning all four rows! - Across these 8 cells: A varies (0, 1), B varies (0, 1), D varies (0, 1). The only variable that remains constant is $C = 0$. - This octet reduces to the single literal: C . - Next, look at the four corners: cells (0, 2, 8, dots) but wait! Look at the top two rows ($AB=00$ and 01): cells (0, 1, 2, 4, 5, 6) and cells (4, 5, 6, 12, 13, 14). - Cells (0, 2, 4, 6) form a Quad: row $A=0$, and columns $CD=00, 10$ ($D=0$). Term: $A D$. - Cells (4, 6, 12, 14) form a Quad: row $B=1$, and columns $CD=00, 10$ ($D=0$). Term: BD . - **Final Minimal SOP Expression**:

$$F = C + A D + B D$$

Notice that an original expression containing 11 four-variable terms (requiring dozens of gates) collapsed into just three tiny 2-literal terms!

2.9 Don't Care Conditions (d or x)

Source: Kumar (Ch. 6, pp. 278–280); Mano (Ch. 3, pp. 116–119)

[Pacing: Conceptually Heavy — A Hardware Designer's Best Friend]

Need (Problem-First)

In many real systems, certain input combinations **can never physically occur**. - *Concrete Example*: A circuit processes BCD digits (0 through 9, binary 0000 to 1001). What happens if the inputs are 1010 (10_{10}) through 1111 (15_{10})? In a legitimate BCD system, the upstream hardware will *never* produce those codes. - Because those input combinations will never happen in real life, **the designer does not care whether the circuit outputs a 0 or a 1 for those rows**.

The Strategic Grouping Rule for Don't Cares (d or x)

A **Don't Care** condition (written as d or x in the K-map) is a wildcard: - **Rule 1**: You may treat an x as a 1 **if and only if it helps you form a larger grouping** (e.g., expanding a pair into a quad, or a quad into an octet). A larger group eliminates more variables, making the circuit cheaper. - **Rule 2**: You treat an x as a 0 (leave it un-circled) **if it does not help enlarge any group of 1s**. You are **never required to cover an x**! Only true 1s are mandatory to cover.

Concrete Worked Example: Minimize: $F(A, B, C, D) = \sum m(1, 3, 7, 11, 15) + \sum d(0, 2, 5)$ - **Analysis:** - Mandatory 1s are at cells **1, 3, 7, 11, 15**. - Don't cares (X) are at cells **0, 2, 5**. - Without don't cares, cells **(3, 7, 11, 15)** form a vertical Quad (term: **CD**). Cell 1 would have to pair with 3 (term: **A BD**). - **With Don't Cares:** By including **X₅**, cells **(1, 3, 5, 7)** form a Quad in the upper half! Even better: cells **(0, 1, 2, 3)** in the top row are all 1s or Xs! By treating **X₀, X₂, X₅** strategically, we group cells **(1, 3, 5, 7)** with **(9, 11, 13, 15)** if present, or group **(3, 7, 11, 15)** as a Quad (**CD**) and group **(1, 3, 5, 7)** as a Quad (**AD**). - **Minimized Result:** $F = CD + AD = D(C + A)$.

Unit II Understanding Checkpoints & Misconception Audit

1. Does This Make Sense? Self-Check

- **Question 1:** Why is a group of three adjacent 1s (3 cells) illegal in a Karnaugh map?
- **Answer:** Because grouping relies on the Boolean theorem $X + X = 1$, which only eliminates a variable when powers of two are factored ($2^1 = 2$, $2^2 = 4$, $2^3 = 8$). A group of 3 cannot factor out a single variable cleanly.
- **Question 2:** If all 16 cells of a 4-variable K-map contain 1s, what is the minimized Boolean function?
- **Answer:** $F = 1$ (the output is permanently HIGH, requiring zero logic gates).
- **Question 3:** Are you required to draw circles around every Don't Care (X) on a K-map?
- **Answer:** Absolutely not! You only include an X if it helps expand a group containing genuine 1s. An X left alone simply defaults to a 0.

2. Conceptual Misconceptions & Arithmetic Slip-Ups Corrected

- **Conceptual Misconception:** "A Karnaugh map can minimize any function with any number of variables."
Correction: K-maps are human graphical tools effective only up to **4 variables** (or awkwardly 5 to 6 variables using stacked 3D layers). For 7 or more variables, visualization collapses and industry uses tabular computer algorithms like the **Quine-McCluskey method** or **Espresso**.
 - **Arithmetic / Notation Slip-Up:** When writing the K-map grid, students constantly label rows or columns as **00, 01, 10, 11**. **This is completely wrong!** The order must be Gray-coded: **00, 01, 11, 10**. Swapping columns 11 and 10 destroys the geometric adjacency and corrupts all resulting terms.
-